Azraq Monte Carlo Risk Engine

Complete API reference

This PDF lists **all HTTP and WebSocket endpoints**, parameters, request/response structure, **worked curl examples**, and **metrics** semantics. **Live Swagger UI:** https://monte-carlo.octaloop.dev/docs#/ — OpenAPI: https://monte-carlo.octaloop.dev/openapi.json

- **Canonical source (edit first):** `docs/ENDPOINTS.md` in the repository.
- **Regenerate this PDF:** `python scripts/build_api_reference_pdf.py` (Markdown + Edge/Chrome headless).

API endpoints — parameters, bodies, responses, usage

Hosted (Swagger UI — try requests in the browser): https://monte-carlo.octaloop.dev/docs#/

Same deployment: OpenAPI JSON — `https://monte-carlo.octaloop.dev/openapi.json` · optional investor dashboard — `https://monte-carlo.octaloop.dev/app/` (if enabled).

Local dev: `http://127.0.0.1:8000` (same paths: `/docs`, `/openapi.json`, `/app/`).

Machine-readable contract: `GET /openapi.json`. **Interactive UI:** `GET /docs` (Swagger).

Longer narrative + risk interpretation: **API.md** (keep this file for the catalogue of routes).

Quick integration flow: **INTEGRATION.md**.

Contents

0. Complete route inventory
1. Authentication & headers
2. Shared JSON models
3. How to send data and read results
4. Endpoint reference
5. Errors
6. Live deployment, examples, and response detail

---

0. Complete route inventory

Application API (implemented in `azraq_mc/api.py`)

There are **20 HTTP operations** and **1 WebSocket** — **21** interfaces total.

#	Method	Path	Summary
1	`GET`	`/health`	Liveness (no API key).
2	`GET`	`/v1/market/yfinance/{symbol}/history`	Yahoo OHLCV (query `period`).
3	`GET`	`/v1/market/yfinance/{symbol}/returns`	Yahoo simple returns.

4	`GET`	`/v1/market/ecb/eur-usd`	ECB EUR/USD spot.
5	`GET`	`/v1/calibration/stress-data-catalog`	Stress-source catalogue.
6	`GET`	`/v1/catalog/full-stack-factors`	12-factor full-stack list.
7	`POST`	`/v1/calibration/preview`	Resolve `dynamic_margins` → `margins` (no MC).
8	`POST`	`/v1/simulate/scheduled/asset`	MC + optional snapshot file.
9	`POST`	`/v1/simulate/asset`	**Main** single-asset Monte Carlo.
10	`POST`	`/v1/simulate/v0/base-case`	Deterministic baseline (body = `AssetAssumptions`).
11	`POST`	`/v1/simulate/portfolio`	Joint MC, ≥2 assets.
12	`POST`	`/v1/shockpack/catalog/register`	Save shock pack → `entry_id`.
13	`POST`	`/v1/shockpack/catalog/{entry_id}/promote`	Change `promotion_tier`.
14	`GET`	`/v1/shockpack/catalog/{entry_id}`	Fetch catalogue row + `spec`.
15	`GET`	`/v1/shockpack/catalog`	List catalogue entries.
16	`POST`	`/v1/shockpack/export/npz`	Write shock `z` to `.npz` on disk.
17	`POST`	`/v1/snapshots/save`	Save prior result JSON.
18	`GET`	`/v1/snapshots/list`	List snapshot paths.
19	`GET`	`/v1/audit/runs`	Recent audit rows (`limit`).
20	`POST`	`/v1/snapshots/diff/asset-metrics`	Diff two asset `SimulationResult` files.
21	`WebSocket`	`/ws/v1/simulate/portfolio`	Portfolio MC + optional progress.

Anything not in this table is **not** a custom business route (see below).

Also served (FastAPI + static UI)

These are automatic or mounted by the app, not separate “risk” APIs:

Kind	Path	Purpose
OpenAPI schema	`GET /openapi.json`	Machine-readable spec (codegen, Postman).
Swagger UI	`GET /docs`	Try-it-out browser UI.
ReDoc	`GET /redoc`	Alternate docs UI (if enabled by FastAPI defaults).
Static dashboard	`GET /app/…`	Investor UI (`frontend/`), only if that folder exists.

---

1. Authentication & headers

Condition	Client behaviour

Server has `AZRAQ_API_KEY` set	Send header `X-API-Key: <same value>` on all routes **except** `GET /health` (health is public).
Optional audit	`X-Azraq-User-Id` — label stored with runs (not authentication).
Catalogue / tenant	`X-Azraq-Tenant-Id` — filters or enforces tenant on some **catalog** `GET` routes.
Promoting shock packs	`POST /v1/shockpack/catalog/{entry_id}/promote` needs `X-Azraq-Catalog-Role` in the allow-list (`AZRAQ_CATALOG_PROMOTER_ROLES`), unless that env is empty (local dev).

WebSocket: if an API key is required, pass header `x-api-key` on the handshake (see portfolio WS below).

---

2. Shared JSON models

These names match **Pydantic / OpenAPI** schemas in the repo. Field-level detail: `/docs` or [API.md](API.md) § "Shared types".

Model	Used for
`FinancingAssumptions`	`debt_principal`, `interest_rate_annual` (decimal), `loan_term_years`, `covenant_dscr`
`AssetAssumptions`	Project economics + `financing` + optional `utility_opex_annual`, `factor_transforms`, `full_stack`, …
`ShockPackSpec`	`shockpack_id`, `seed`, `n_scenarios`, `sampling_method`, `margins`, optional `dynamic_margins`, `correlation`, `factor_order`, …
`AdhocSimulationRequest`	`asset` + `shockpack` and/or `shockpack_catalog_entry_id` + optional attribution / `performance_profile`
`ScheduledAssetRequest`	Same as adhoc + `label`, `persist`, `model_version`
`PortfolioSimulationRequest`	`portfolio_id`, `portfolio_assumption_set_id`, `assets` (array, **≥ 2**), `shockpack` or catalogue id
`SimulationResult`	`metadata`, `metrics`, optional `attribution`, `full_stack`, `extensions`
`BaseCaseResult`	Deterministic run: `metadata`, `base` (point estimates)
`PortfolioSimulationResult`	`metadata`, `per_asset[]`, `portfolio`
`CalibrationPreviewResponse`	`resolved_shockpack`, `calibration_trace`
`ScheduledAssetResponse`	`result` (`SimulationResult`), `snapshot_path`

Most JSON responses also include `response_help` (plain-language hints).

---

3. How to send data and read results

Single-project Monte Carlo (typical integration)

1. **Build** an `AssetAssumptions` object from your systems (revenue, opex, capex, debt, rates, covenant, horizon, ids).
2. **Build** a `ShockPackSpec` (at least `shockpack_id`, `seed`, `n_scenarios`, `sampling_method`, and usually `margins` or `dynamic_margins` for calibration).

3. **POST** `/v1/simulate/asset` with body `{ "asset": {...}, "shockpack": {...}, ... }`.
4. **Read** `200` JSON `SimulationResult`: use `metrics.dscr`, `metrics.irr_annual`, `metrics.covenant_breach_probability`, tail fields, etc., and `metadata` for `run_id`, timing, seed.

State: there is **no session**. Each POST is independent. To "reset," send a new body.

Optional: validate calibration only (no MC)

POST `/v1/calibration/preview` with a full `ShockPackSpec` (e.g. with `dynamic_margins.yahoo_finance`). Response gives `resolved_shockpack` with concrete `margins` — paste that into simulate.

Optional: save results for diffing

- **POST** `/v1/simulate/scheduled/asset` with `persist: true` → get `snapshot_path`.
- Or **POST** `/v1/snapshots/save` with a prior `SimulationResult` in `result`.

Then **POST** `/v1/snapshots/diff/asset-metrics` with `before_path` / `after_path`.

---

4. Endpoint reference

Legend: **Path** / **Query** / **Body** = where parameters go. **Auth** = needs `X-API-Key` when server key is set (except `/health`).

`GET /health`

Purpose	Liveness: server responds.
Parameters	None.
Response	`{ "status": "ok", "response_help": { ... } }`
Usage	Monitoring or pre-flight check. **No model run.**
Auth	None.

`GET /v1/market/yfinance/{symbol}/history`

Purpose	Raw OHLCV history from Yahoo (inspection / reporting).
Path	`symbol` — ticker, e.g. `HG=F`.
Query	`period` — `1d`, `5d`, `1mo`, `3mo`, `6mo`, `1y`, `2y`, `5y`, `10y`, `ytd`, `max` (default `1y`).
Response	`{ "symbol", "period", "data": [ ... ] }` + `response_help`. `404` if no data; `501` if `yfinance` missing.
Usage	Does **not** run Monte Carlo. For feeding vol into the engine use `dynamic_margins.yahoo_finance` or `/v1/calibration/preview`.
Auth	Optional key.

`GET /v1/market/yfinance/{symbol}/returns`

Purpose	Close-to-close **simple** returns series (human review).
Path	`symbol`.
Query	`period` — same set as history.
Response	`{ "symbol", "period", "returns": [{"date","return"},...], "n" }` + `response_help`. `404` / `501` as above.
Usage	Exploratory; internal calibration uses **log** returns + annualisation.
Auth	Optional key.

`GET /v1/market/ecb/eur-usd`

Purpose	ECB daily **EUR/USD** reference from public XML.
Parameters	None.
Response	Rate fields + `response_help`. `502` if feed fails.
Usage	FX sanity checks; wire into `dynamic_margins` via your own JSON/file/http if needed.
Auth	Optional key.

`GET /v1/calibration/stress-data-catalog`

Purpose	Catalogue of stress / data sources and integration hints (Appendix J–style).
Parameters	None.
Response	`{ "sources", "integration_overview", "response_help" }`.
Usage	Discover URLs and how they map to `dynamic_margins`.
Auth	Optional key.

`GET /v1/catalog/full-stack-factors`

Purpose	Ordered `factor_order` + metadata for **12-factor full-stack** mode.
Parameters	None.
Response	`{ "factor_order", "risk_factors", "response_help" }`.
Usage	Required before `asset.full_stack` + matching 12-factor `ShockPackSpec`.
Auth	Optional key.

`POST /v1/calibration/preview`

Purpose	Resolve `dynamic_margins` (file / http / Yahoo) into numeric `margins` **without** drawing scenarios.
Body	`ShockPackSpec` (JSON) — same shape as `shockpack` on simulate.
Response	`CalibrationPreviewResponse`: `resolved_shockpack`, `calibration_trace`.
Usage	Governance: confirm `resolved_shockpack.margins` before expensive `/v1/simulate/asset`.
Auth	Optional key.

`POST /v1/simulate/v0/base-case`

Purpose	**Deterministic** single path (no random shocks). Fast tie-out vs spreadsheet.
Body	`AssetAssumptions` as **root** JSON (not wrapped in `{ "asset": ... }`).
Response	`BaseCaseResult`: `metadata` (`execution_mode: "v0_base"`), `base` point estimates (`dscr`, `irr_annual`, ...).
Usage	Baseline only — **not** tail risk.
Auth	Optional key.

`POST /v1/simulate/asset`

Purpose	Main **Monte Carlo** for **one** asset — DSCR/IRR distributions, breach probability, etc.
Body	`AdhocSimulationRequest`: `asset` (required); `shockpack` and/or `shockpack_catalog_entry_id`; optional `include_attribution`, `include_advanced_attribution`, `attribution_tail_fraction`, `performance_profile`.
Response	`SimulationResult`: `metadata`, `metrics`, optional `attribution`, `full_stack`, `extensions`.
Usage	Primary integration: **send** economics + shock spec, **receive** `metrics` for dashboards or storage.
Auth	Optional key.

`POST /v1/simulate/scheduled/asset`

Purpose	Same maths as `/v1/simulate/asset`, optional **on-disk JSON snapshot** for monitoring / diff.
Body	`ScheduledAssetRequest`: all `AdhocSimulationRequest` fields + `label`, `persist` (default true), `model_version`.
Response	`ScheduledAssetResponse`: `result` (`SimulationResult`), `snapshot_path` or `null`.
Usage	Recurring runs; pair with `/v1/snapshots/diff/asset-metrics`. Snapshot root: `AZRAQ_SNAPSHOT_DIR` or default `data/snapshots`.
Auth	Optional key.

`POST /v1/simulate/portfolio`

Purpose	**Joint** Monte Carlo for **≥ 2** assets (same shock draw per scenario across names).
Body	`PortfolioSimulationRequest`: `shockpack` or catalogue id; `portfolio_id`, `portfolio_assumption_set_id`, `assets` (array length ≥ 2); optional `performance_profile`.
Response	`PortfolioSimulationResult`: `metadata`, `per_asset`, `portfolio` (e.g. `probability_any_covenant_breach`, `min_dscr_across_assets`).
Usage	Portfolio concentration; joint breach ≠ sum of silo runs.
Auth	Optional key.

`WebSocket` `WS /ws/v1/simulate/portfolio`

Purpose	Same as `POST /v1/simulate/portfolio` with optional **progress** messages.
Handshake	If `AZRAQ_API_KEY` set: header `x-api-key`.
Messages	Client sends **one** JSON = `PortfolioSimulationRequest`. Server: optional `{ "type":"progress", ... }`, then `{ "type":"result", "body": <PortfolioSimulationResult> }` or `{ "type":"error", "detail" }`.
Usage	Long runs from browser / clients that prefer streaming.
Auth	`x-api-key` when server key is set.

`POST /v1/shockpack/catalog/register`

Purpose	Store a `ShockPackSpec` in the catalogue for reuse.
Body	`ShockPackSpec` (JSON).
Query	`semver` (default `1.0.0`), optional `tenant_id`, `promotion_tier` (`dev`/`staging`/`prod`), `rbac_owner_role`.
Response	`{ "entry_id": "<uuid>", "response_help": ... }` — use `entry_id` as `shockpack_catalog_entry_id` on simulate.
Usage	Freeze governance-approved shock design; inline `shockpack` can still patch fields.
Auth	Optional key.

`POST /v1/shockpack/catalog/{entry_id}/promote`

Purpose	Change catalogue `promotion_tier` (e.g. dev → prod).
Path	`entry_id`.
Query	`to_tier` — `dev` | `staging` | `prod`.
Headers	`X-Azraq-Catalog-Role` unless promoter env list is empty.
Response	`{ "ok": true, "response_help": ... }` — `403` if role not allowed.

Usage	Gate “approved” packs in `GET /v1/shockpack/catalog?promotion_tier=prod`.
Auth	Optional key + promoter role.

`GET /v1/shockpack/catalog/{entry_id}`

Purpose	Fetch one catalogue row including full `spec`.
Path	`entry_id`.
Headers	Optional `X-Azraq-Tenant-Id` — must match entry tenant if enforced.
Response	Catalogue object + `response_help`. `403` tenant mismatch; `404` missing.
Usage	Inspect exact shock JSON before production runs.
Auth	Optional key.

`GET /v1/shockpack/catalog`

Purpose	List recent registrations (**metadata**; use **GET by id** for full `spec`).
Query	`limit` (default 100), optional `tenant_id`, `promotion_tier`. Header tenant may filter.
Response	`{ "entries": [ ... ], "response_help": ... }`.
Usage	Discover `entry_id` for CI/CD.
Auth	Optional key.

`POST /v1/shockpack/export/npz`

Purpose	Write correlated shock array `Z` to a `.npz` file on the server (offline HPC / custom models).
Body	`ShockExportRequest`: `shockpack` (required), optional `directory`.
Response	`{ "path": "<absolute .npz path>", "response_help": ... }`. `dynamic_margins` resolved before draw.
Usage	No DSCR/IRR — only random draws. Default dir: `AZRAQ_SHOCK_EXPORT_DIR` or `data/shockpacks`.
Auth	Optional key.

`POST /v1/snapshots/save`

Purpose	Persist an **already computed** result to JSON on disk.
Body	`SnapshotSaveRequest`: optional `label`, `result` = full `SimulationResult` / `PortfolioSimulationResult` / `BaseCaseResult`.
Response	`{ "path": "<absolute path>", "response_help": ... }`.

Usage	Bookmark or feed external tools; same root as scheduled snapshots.
Auth	Optional key.

`GET /v1/snapshots/list`

Purpose	List snapshot **file paths** under the snapshot root.
Parameters	None.
Response	`{ "paths": [ "..." ], "response_help": ... }`.
Usage	Pick `before_path` / `after_path` for diff.
Auth	Optional key.

`POST /v1/snapshots/diff/asset-metrics`

Purpose	Compare two **saved** asset `SimulationResult` snapshots.
Body	`{ "before_path": "<server path>", "after_path": "<server path>" }`.
Response	`metrics_delta`, `provenance_delta`, `response_help`.
Usage	Explain drift: assumptions vs shock vs seed vs model version. `400` if either file is not an asset MC result.
Auth	Optional key.

`GET /v1/audit/runs`

Purpose	Recent simulation events from SQLite audit DB (if enabled).
Query	`limit` (default 50).
Response	`{ "runs": [ { "run_id", "run_kind", "asset_id", "shockpack_id", "seed", "n_scenarios", ... } ], "response_help": ... }`.
Usage	Correlate `run_id` with `SimulationResult.metadata.run_id`. Empty if audit disabled/missing.
Auth	Optional key.

---

5. Errors

HTTP	Typical cause
401	Missing or wrong `X-API-Key` when server requires it.
403	Catalogue tenant / promoter role mismatch.

404	Yahoo history missing; unknown catalogue `entry_id`.
400	Validation: bad JSON shape, wrong asset count for portfolio, wrong snapshot types for diff.
422	Pydantic validation detail (FastAPI) — check `detail` array for field paths.
501	Optional dependency missing (e.g. `yfinance`).
502	Upstream feed failure (e.g. ECB).

For `422`, use `/docs` "Try it out" or client validation against `/openapi.json` to align payloads before production.

---

6. Live deployment, examples, and response detail

6.1 Public host (Octaloop)

Resource	URL
Swagger UI (schemas + "Try it out")	https://monte-carlo.octaloop.dev/docs#/
OpenAPI JSON (import to Postman, codegen)	`https://monte-carlo.octaloop.dev/openapi.json`
Web app (if deployed)	`https://monte-carlo.octaloop.dev/app/`

Use `https://monte-carlo.octaloop.dev` as the host for the paths in §4 (e.g. `POST https://monte-carlo.octaloop.dev/v1/simulate/asset`). If the server has `AZRAQ_API_KEY` set, add header `X-API-Key` on calls other than `GET /health`.

6.2 `AdhocSimulationRequest` — request fields (main Monte Carlo)

Field	Required	Type / notes
`asset`	Yes	Full `AssetAssumptions` (ids, horizon, revenue, opex, capex, equity, financing, optional `utility_opex_annual`, `factor_transforms`, `full_stack`, ...).
`shockpack`	One of pack or catalogue	Inline `ShockPackSpec` (`shockpack_id`, `seed`, `n_scenarios`, `sampling_method`, `margins`, optional `dynamic_margins`, `correlation`, ...).
`shockpack_catalog_entry_id`	One of pack or catalogue	UUID from `POST /v1/shockpack/catalog/register`; inline `shockpack` can patch fields.
`include_attribution`	No	Default `false`. Tail attribution (indicative narrative).
`include_advanced_attribution`	No	Default `false`. Extra factor weights when attribution on.
`attribution_tail_fraction`	No	Default `0.05` (e.g. worst ~5% scenarios for attribution).
`performance_profile`	No	`"interactive"` (caps `n_scenarios`, good for UI), `"standard"`, `"deep"` (no cap), or `null`.

6.3 `SimulationResult.metrics` — main outputs (read after `POST /v1/simulate/asset`)

Field	Meaning
`dscr`	**`DistributionSummary`**: percentiles (e.g. `p05`–`p95`), `mean`, `std` — debt service coverage over the run.
`irr_annual`	**`DistributionSummary`** for equity IRR (decimal per year, e.g. `0.08` = 8%).
`total_capex`	Optional **`DistributionSummary`** for stochastic nominal build cost when the engine supplies it.
`covenant_breach_probability`	Share of paths where modeled DSCR falls below **`asset.financing.covenant_dscr`**.
`probability_of_default_proxy_dscr_lt_1`	PD-style proxy using DSCR < 1 (see OpenAPI / product definition).
`var_irr_95` / `cvar_irr_95`	Tail / shortfall style metrics on IRR when populated.
`ebitda`, `levered_cf`, `nav_proxy_equity`, …	Other **`DistributionSummary`** blocks when enabled by model path.

Always read **`metadata`** for **`run_id`**, **`seed`**, **`n_scenarios`**, **`compute_time_ms`**, **`shockpack_id`**, optional **`margin_calibration_trace`** (if Yahoo/file/http calibration ran).

6.4 Example: health check (no API key)

Replace `HOST` with `https://monte-carlo.octaloop.dev` or `http://127.0.0.1:8000`.

```
curl -sS "${HOST}/health"
```

6.5 Example: single-asset Monte Carlo (minimal body)

Send **`Content-Type: application/json`**. If an API key is required, add `-H "X-API-Key: YOUR_KEY"`.

```
curl -sS -X POST "${HOST}/v1/simulate/asset" \
  -H "Content-Type: application/json" \
  -d '{
    "shockpack": {
      "shockpack_id": "demo-sp-1",
      "seed": 42,
      "n_scenarios": 800,
      "sampling_method": "monte_carlo",
      "margins": {
        "revenue_log_mean": 0,
        "revenue_log_sigma": 0.08,
        "capex_log_mean": 0,
        "capex_log_sigma": 0.06,
        "opex_log_mean": 0,
        "opex_log_sigma": 0.05,
        "rate_shock_sigma": 0.005
      }
    },
    "asset": {
      "asset_id": "demo-asset-1",
      "assumption_set_id": "demo-as-1",
      "horizon_years": 8,
```

```
      "base_revenue_annual": 12000000,
      "base_opex_annual": 5000000,
      "initial_capex": 80000000,
      "equity_fraction": 0.35,
      "tax_rate": 0.0,
      "financing": {
        "debt_principal": 40000000,
        "interest_rate_annual": 0.055,
        "loan_term_years": 12,
        "covenant_dscr": 1.2
      }
    },
    "include_attribution": false,
    "performance_profile": "interactive"
  }'
```

Note: The JSON above uses a fully specified `margins` block. If `dynamic_margins` (Yahoo/file/http) is used instead, run `POST /v1/calibration/preview` first to validate resolved sigmas. Omit `performance_profile` or use `"standard"` / `"deep"` for higher scenario limits (see OpenAPI).

6.6 Example: deterministic baseline (no Monte Carlo)

Body is `AssetAssumptions` only (not wrapped in `asset`).

```
curl -sS -X POST "${HOST}/v1/simulate/v0/base-case" \
  -H "Content-Type: application/json" \
  -d '{"asset_id":"demo","assumption_set_id":"v0","horizon_years":8,
       "base_revenue_annual":12000000,"base_opex_annual":5000000,
       "initial_capex":80000000,"equity_fraction":0.35,"tax_rate":0,
       "financing":{"debt_principal":40000000,"interest_rate_annual":0.055,
       "loan_term_years":12,"covenant_dscr":1.2}}'
```

6.7 WebSocket portfolio run

Connect to `wss://monte-carlo.octaloop.dev/ws/v1/simulate/portfolio` (or `ws://127.0.0.1:8000/...` locally). Send **one** message: same JSON as `POST /v1/simulate/portfolio`. If API key is set, send header `x-api-key` on the handshake.

6.8 Where to see every field

Open **Swagger UI**, expand `POST /v1/simulate/asset`, inspect **Request body** and **Response** schemas (`AdhocSimulationRequest`, `SimulationResult`, `FinancialRiskMetrics`, ...). The PDF cannot replace the live schema explorer for nested optional fields.

Related documents

File	Role
API.md	Full narrative, examples, `response_help`, security notes
INTEGRATION.md	Minimal request, mapping checklist, stateless “reset”

PROJECT_OVERVIEW.md	Product-level engine description
`Azraq_Monte_Carlo_API_Reference.pdf`	Print-ready PDF of this reference (`python scripts/build_api_reference_pdf.py`)